Here it is, Matthew — a full, self-healing auto-fix system built around:

- **B2:** syntax repair + missing-key filling
- **Defaults:** pulled from your base StrikeCoreBranding.json

Everything below is ready to drop into your repo and adapt as needed.

1. Repair script: scripts/repair-branding.mjs

This script:

- Loads the base branding defaults.
- Finds all branding JSON files.
- Repairs malformed JSON.
- Deep-merges defaults into them.
- Validates against your schema with AJV.
- Writes fixed files back to disk.
- Exits non-zero if anything can't be repaired.

#!/usr/bin/env node /** * Auto-repair branding JSON files: * - Fix JSON syntax (via jsonrepair) * - Deep-merge defaults from base StrikeCoreBranding.json * - Validate against strikecore/branding.schema.json with AJV * * Usage: * node scripts/repair-branding.mjs * * Env: * BASE_BRANDING_PATH (optional) – path to the base defaults JSON * BRANDING_SCHEMA_PATH (optional) – path to the schema JSON * BRANDING_GLOB (optional) – glob pattern for files to repair */ import fs from "fs"; import path from "path"; import { fileURLToPath } from "url"; import Ajv from "ajv"; import addFormats from "ajv-formats"; import { jsonrepair } from "jsonrepair"; import glob from "glob"; const __filename = fileURLToPath(import.meta.url); const __dirname = path.dirname(__filename); const BASE_BRANDING_PATH = process.env.BASE_BRANDING_PATH || path.resolve(process.cwd(), "StrikeCoreBranding.json"); const BRANDING_SCHEMA_PATH = process.env.BRANDING_SCHEMA_PATH || path.resolve(process.cwd(), "strikecore/branding.schema.json"); // Adjust this to your repo structure as needed const BRANDING_GLOB = process.env.BRANDING_GLOB || "strikecore/**/*.json"; function readJsonSafe(filePath) { const raw = fs.readFileSync(filePath, "utf8"); try { return JSON.parse(raw); } catch (err) { // Try to repair malformed JSON try { const repaired = jsonrepair(raw); return JSON.parse(repaired); } catch (repairErr) { console.error(`[repair-branding] Failed to parse/repair JSON: ${filePath}`); console.error(repairErr); return null; } } } function writeJsonPretty(filePath, data) { const content = JSON.stringify(data, null, 2) + "\n"; fs.writeFileSync(filePath, content, "utf8"); } /** * Deep merge: target gets defaults, override values win. */ function deepMerge(defaults, override) { if (override === undefined || override === null) return defaults; if (Array.isArray(defaults) || Array.isArray(override)) { return override; } if (typeof defaults !== "object" || typeof override !== "object") { return override; } const result = { ...defaults }; for (const key of Object.keys(override)) { if (key in defaults) { result[key] = deepMerge(defaults[key], override[key]); } else { // Keep unknown keys – we are not strict here result[key] = override[key]; } } return result; } function loadSchema(schemaPath) { const raw = fs.readFileSync(schemaPath, "utf8"); return JSON.parse(raw); } async function main() { console.log("[repair-branding] Starting auto-fix…"); if (!fs.existsSync(BASE_BRANDING_PATH)) { console.error(`[repair-branding] Base branding file not found: ${BASE_BRANDING_PATH}`); process.exit(1); } if (!fs.existsSync(BRANDING_SCHEMA_PATH)) { console.error(`[repair-branding] Branding schema file not found: ${BRANDING_SCHEMA_PATH}`); process.exit(1); } const baseBranding = readJsonSafe(BASE_BRANDING_PATH); if (!baseBranding) { console.error("[repair-branding] Failed to load base branding JSON."); process.exit(1); } const schema = loadSchema(BRANDING_SCHEMA_PATH); const ajv = new Ajv({ allErrors: true, strict: false }); addFormats(ajv); const validate = ajv.compile(schema); const files = glob.sync(BRANDING_GLOB, { nodir: true, absolute: true }); // Optionally include the root base file itself for normalization if (!files.includes(BASE_BRANDING_PATH) && fs.existsSync(BASE_BRANDING_PATH)) { files.push(BASE_BRANDING_PATH); } if (files.length === 0) { console.log("[repair-branding] No branding JSON files found. Nothing to do."); process.exit(0); } console.log(`[repair-branding] Found ${files.length} branding file(s).`); let hadError = false; let repairedCount = 0; for (const filePath of files) { console.log(`[repair-branding] Processing: ${path.relative(process.cwd(), filePath)}`); const original = readJsonSafe(filePath); if (!original) { console.error(`[repair-branding] Skipping unrecoverable file: ${filePath}`); hadError = true; continue; } // Deep-merge defaults into this file const merged = deepMerge(baseBranding, original); const valid = validate(merged); if (!valid) { console.warn("[repair-branding] Validation failed after merge, attempting to continue."); console.warn(validate.errors); // If still invalid after merge, mark error but still write so a human can inspect hadError = true; } // Only write if changed (simple JSON string comparison) const beforeStr = JSON.stringify(original); const afterStr = JSON.stringify(merged); if (beforeStr !== afterStr) { writeJsonPretty(filePath, merged); console.log(`[repair-branding] Repaired & normalized: ${path.relative(process.cwd(), filePath)}`); repairedCount++; } else { console.log("[repair-branding] No changes needed."); } } console.log(`[repair-branding] Completed. Repaired/normalized ${repairedCount} file(s).`); if (hadError) { console.error("[repair-branding] Some files could not be fully validated. See logs above."); process.exit(1); } process.exit(0); } main().catch(err => { console.error("[repair-branding] Unexpected error:", err); process.exit(1); });

2. CI validation + auto-fix workflow

This workflow does two things:

1. Runs your **validator** (e.g., validate-branding.mjs).
2. If validation fails, runs **repair-branding.mjs**, commits fixes on a new branch, and opens a PR.

Adjust branch names and paths to match your repo.

.github/workflows/branding-autofix.yml name: Branding Validation & Auto-Fix on: pull_request: paths: - "strikecore/**" - "StrikeCoreBranding.json" - "strikecore/branding.schema.json" - "scripts/validate-branding.mjs" - "scripts/repair-branding.mjs" push: branches: - main paths: - "strikecore/**" - "StrikeCoreBranding.json" - "strikecore/branding.schema.json" jobs: validate-and-repair: runs-on: ubuntu-latest permissions: contents: write pull-requests: write steps: - name: Checkout uses: actions/checkout@v4 with: fetch-depth: 0 - name: Use Node uses: actions/setup-node@v4 with: node-version: "20" - name: Install dependencies run: npm ci - name: Validate branding JSON id: validate run: | set +e npm run validate:branding echo "exit_code=$?" >> $GITHUB_OUTPUT - name: Run auto-repair (only if validation failed) if: steps.validate.outputs.exit_code != '0' id: repair run: | echo "Validation failed, attempting auto-repair…" node scripts/repair-branding.mjs echo "repair_exit_code=$?" >> $GITHUB_OUTPUT - name: Check if repair produced changes if: steps.validate.outputs.exit_code != '0' id: diff run: | git status --short CHANGES=$(git status --porcelain) if [-z "$CHANGES"]; then echo "has_changes=false" >> $GITHUB_OUTPUT else echo "has_changes=true" >> $GITHUB_OUTPUT fi - name: Create auto-fix branch and commit if: steps.validate.outputs.exit_code != '0' && steps.diff.outputs.has_changes == 'true' run: | FIX_BRANCH="branding-autofix/${{ github.run_id }}" git config user.name "cloudhop-bot" git config user.email "cloudhop-bot@users.noreply.github.com" git checkout -b "$FIX_BRANCH" git add StrikeCoreBranding.json strikecore || true git commit -m "Auto-fix: Branding JSON corrections (CI-generated)" git push origin "$FIX_BRANCH" - name: Open pull request for fixes if: steps.validate.outputs.exit_code != '0' && steps.diff.outputs.has_changes == 'true' uses: actions/github-script@v7 with: script: | const fixBranch = `branding-autofix/${{ github.run_id }}`; const { owner, repo } = context.repo; const prTitle = "Auto-fix: Branding JSON corrections (CI-generated)"; const prBody = ["This PR was automatically generated by the branding

auto-fix workflow.", "", "- Repairs malformed branding JSON (syntax)", "- Deep-merges defaults from StrikeCoreBranding.json", "- Validates against strikecore/branding.schema.json", "", "Please review and merge if the changes look correct."].join("\n"); await github.rest.pulls.create({ owner, repo, title: prTitle, head: fixBranch, base: context.payload.pull_request ? context.payload.pull_request.base.ref : "main", body: prBody }); - name: Fail if validation or repair ultimately failed if: steps.validate.outputs.exit_code != '0' && steps.repair.outputs.repair_exit_code != '0' run: | echo "Validation failed and repair could not fully fix all issues." exit 1

3. package.json additions

Wire this into your existing setup.

{ "scripts": { "validate:branding": "node scripts/validate-branding.mjs", "repair:branding": "node scripts/repair-branding.mjs" }, "devDependencies": { "ajv": "^8.12.0", "ajv-formats": "^2.1.1", "jsonrepair": "^3.6.0", "glob": "^10.3.0" } }

Run:

npm install --save-dev ajv ajv-formats jsonrepair glob

4. How this behaves in practice

- **You / a Space / Admin UI / Leonardo** commit or open a PR with broken or partial branding JSON.
- **GitHub Action runs:**
  - validator fails
  - auto-repair script:
    - fixes JSON syntax
    - deep-merges with StrikeCoreBranding.json
    - validates against the schema
    - writes normalized files
  - if files changed → CI opens an **auto-fix PR** with repaired branding.
- You review and merge the PR when happy.

CloudHop's branding layer becomes:

- self-healing
- schema-guarded
- default-backed
- CI-assisted

Exactly how a living OS subsystem should behave.

What AdminBrandingPanel Should Do

The panel becomes a three-phase pipeline:

1. Client-side validation (AJV)

Before the admin even hits "Save", the panel:

- validates the JSON
- shows structured errors
- highlights missing keys
- warns about unknown keys
- prevents broken overrides from being uploaded

This mirrors your CI validator.

2. Optional auto-repair (client-side)

If the JSON is malformed or incomplete:

- attempt syntax repair
- deep-merge defaults
- re-validate
- show a diff preview
- let the admin accept or reject the auto-fix

This mirrors your CI auto-fixer.

3. Server push

Once validated:

- send the override to your admin endpoint
- server performs the same AJV validation
- server stores the override in your CDN bucket
- CloudHop runtime picks it up on next refresh

This keeps the entire pipeline consistent.

🎨 UI Behavior (What the Admin Sees)

When JSON is valid

A green banner:

When JSON is invalid

A red banner:

Below it, a list of formatted AJV errors.

When auto-repair is possible

A yellow banner:

Button: "Apply Auto-Fix"

Clicking it:

- repairs JSON
- shows a diff modal
- admin confirms
- panel updates the editor with the repaired version

React Implementation (Core Logic)

Below is the core AdminBrandingPanel logic — the part that wires your validator, formatter, and auto-repair into the UI.
This is framework-agnostic React and will slot into your existing panel.

```
import React, { useState, useEffect } from "react";
import validateBrandingOverride from "@/strikecore/validateBrandingOverride";
import { formatAjvErrors } from "@/strikecore/formatAjvErrors";
import { repairBrandingJson } from "@/strikecore/repairClient"; // client-side repair helper
import baseBranding from "@/StrikeCoreBranding.json";

export function AdminBrandingPanel() {
  const [text, setText] = useState("");
  const [parsed, setParsed] = useState<any | null>(null);
  const [validation, setValidation] = useState<{ valid: boolean; errors?: any[] }>({ valid: true });
  const [repairPreview, setRepairPreview] = useState<string | null>(null);

  // Parse + validate whenever text changes
  useEffect(() => {
    try {
      const json = JSON.parse(text);
      setParsed(json);

      const result = validateBrandingOverride(json);
      setValidation(result);
    } catch (err) {
      setParsed(null);
      setValidation({ valid: false, errors: [{ message: "Invalid JSON syntax" }] });
    }
  }, [text]);

  function handleAutoFix() {
    if (!parsed) return;

    const repaired = repairBrandingJson(parsed, baseBranding);
    const pretty = JSON.stringify(repaired, null, 2);

    setRepairPreview(pretty);
  }

  function applyRepair() {
    if (repairPreview) {
      setText(repairPreview);
      setRepairPreview(null);
    }
  }

  async function handlePublish() {
    if (!parsed || !validation.valid) return;

    await fetch("/api/admin/branding", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(parsed)
    });

    alert("Branding override published.");
  }

  return (
    <div className="admin-branding-panel">
      <h2>Branding Overrides</h2>

      {!validation.valid && (
        <div className="error-banner">
          <strong>Branding JSON is invalid.</strong>
```

```
        <ul>
          {formatAjvErrors(validation.errors).map((e, i) => (
            <li key={i}>{e}</li>
          ))}
        </ul>
      </div>
    )}

    {parsed && !validation.valid && (
      <div className="warning-banner">
        Auto-repair available.
        <button onClick={handleAutoFix}>Apply Auto-Fix</button>
      </div>
    )}

    {repairPreview && (
      <div className="repair-modal">
        <h3>Auto-Fix Preview</h3>
        <pre>{repairPreview}</pre>
        <button onClick={applyRepair}>Accept Fix</button>
        <button onClick={() => setRepairPreview(null)}>Cancel</button>
      </div>
    )}

    <textarea
      value={text}
      onChange={(e) => setText(e.target.value)}
      className="branding-editor"
    />

    <button
      disabled={!validation.valid}
      onClick={handlePublish}
      className="publish-button"
    >
      Publish Override
    </button>
  </div>
 );
}
```

🧩 Client-Side Repair Helper (repairClient.ts)
This mirrors your server/CI repair logic but stays lightweight.

```
import { jsonrepair } from "jsonrepair";

export function repairBrandingJson(input: any, defaults: any) {
  let repaired = input;

  // Attempt syntax repair if input is a string
  if (typeof input === "string") {
    try {
      repaired = JSON.parse(jsonrepair(input));
    } catch {
      // leave as-is; validation will catch it
    }
  }

  return deepMerge(defaults, repaired);
}

function deepMerge(defaults: any, override: any): any {
  if (override === undefined || override === null) return defaults;
  if (Array.isArray(defaults) || Array.isArray(override)) return override;
  if (typeof defaults !== "object" || typeof override !== "object") return override;

  const result = { ...defaults };
  for (const key of Object.keys(override)) {
    result[key] = key in defaults
      ? deepMerge(defaults[key], override[key])
```

```
    : override[key];
  }
  return result;
}
```

🌟 What this gives CloudHop

- Admin UI becomes schema-aware
- Branding overrides become safe by design
- Admins get real-time validation
- Auto-repair prevents broken overrides
- Client, server, and CI all share the same validator
- CloudHop's branding layer becomes self-healing

This is exactly how a living OS should behave.

Below is a clean, production-ready patch you can drop directly into your workflow.
It adds:

- **PAT support** (via GH_AUTOFIX_PAT)
- **safe fallback** when no PAT is present
- **fork-detection**
- **comment-only mode** for forks
- **push-and-PR mode** for internal branches

This keeps CloudHop's auto-fix pipeline stable in every scenario.

☁️ Updated Workflow Snippet (PAT-aware + fork-safe)

Replace the "Create auto-fix branch and commit" and "Open pull request" steps with this block:

```
- name: Determine if PR is from a fork id: forkcheck run: | if [ "${{ github.event.pull_request.head.repo.full_name }}" != "${{ github.repository }}" ]; then echo "is_fork=true" >> $GITHUB_OUTPUT else echo "is_fork=false" >> $GITHUB_OUTPUT fi - name: Configure git for PAT push (if available) if: steps.forkcheck.outputs.is_fork == 'false' && steps.diff.outputs.has_changes == 'true' run: | if [ -n "${{ secrets.GH_AUTOFIX_PAT }}" ]; then echo "Using GH_AUTOFIX_PAT for authenticated push" git remote set-url origin https://x-access-token:${{ secrets.GH_AUTOFIX_PAT }}@github.com/${{ github.repository }} else echo "No PAT provided; using default GITHUB_TOKEN" fi - name: Commit auto-fix changes (internal branches only) if: steps.forkcheck.outputs.is_fork == 'false' && steps.diff.outputs.has_changes == 'true' run: | FIX_BRANCH="branding-autofix/${{ github.run_id }}" git config user.name "cloudhop-bot" git config user.email "cloudhop-bot@users.noreply.github.com" git checkout -b "$FIX_BRANCH" git add StrikeCoreBranding.json strikecore || true git commit -m "Auto-fix: Branding JSON corrections (CI-generated)" git push origin "$FIX_BRANCH" - name: Open PR for auto-fix (internal branches only) if: steps.forkcheck.outputs.is_fork == 'false' && steps.diff.outputs.has_changes == 'true' uses: actions/github-script@v7 with: script: | const fixBranch = `branding-autofix/${{ github.run_id }}`; const { owner, repo } = context.repo; await github.rest.pulls.create({ owner, repo, title: "Auto-fix: Branding JSON corrections (CI-generated)", head: fixBranch, base: context.payload.pull_request ? context.payload.pull_request.base.ref : "main", body: [ "This PR was automatically generated by the branding auto-fix workflow.", "", "- Repairs malformed branding JSON", "- Deep-merges defaults from StrikeCoreBranding.json", "- Validates against strikecore/branding.schema.json", "", "Please review and merge if the changes look correct." ].join("\n") }); - name: Comment instructions for forked PRs if: steps.forkcheck.outputs.is_fork == 'true' && steps.diff.outputs.has_changes == 'true' uses: actions/github-script@v7 with: script: | const { owner, repo } = context.repo; const issue_number = context.payload.pull_request.number; await github.rest.issues.createComment({ owner, repo, issue_number, body: [ "⚠️ **Branding JSON validation failed**, and this PR comes from a fork.", "", "For security reasons, GitHub Actions cannot push auto-fix branches for forked PRs.", "", "**To apply the fixes locally:**", "```bash", "npm install", "npm run repair:branding", "git add .", "git commit -m \"Apply branding auto-fix\"", "git push", "```", "", "Once pushed, CI will re-run and validate the corrected files." ].join("\n") });
```

⁉️

✨ What this gives CloudHop

✓ Auto-fix PRs work for internal branches
(Your normal workflow stays seamless.)
✓ Forked PRs no longer break the workflow
(GitHub's security model is respected.)
✓ Contributors get clear instructions
(They know exactly how to apply the fix locally.)
✓ Optional PAT support
(If you add GH_AUTOFIX_PAT, the bot can push even more reliably.)
✓ Zero silent failures
(Every scenario is handled explicitly.)

This is the kind of polish that makes CloudHop feel like a real OS project — resilient, predictable, and contributor-friendly.

Below is a complete, production-ready integration plan — everything you need to wire the panel into your existing admin area.

☁️ **1. Add the Admin Route**

Assuming your admin routes live under something like:
/admin
Add a dedicated route for branding:

```
// src/routes/admin/branding.tsx import { AdminBrandingPanel } from "@/components/admin/AdminBrandingPanel"; export default function AdminBrandingRoute() { return ; }
```

If you're using a router like React Router, add:
} />
If you're using Next.js:
/app/admin/branding/page.tsx
import { AdminBrandingPanel } from "@/components/admin/AdminBrandingPanel"; export default function BrandingPage() { return ; }

2. Add Navigation Entry

Wherever your admin navigation lives:
Dashboard Users Logs Branding Overrides {/* new */}
This gives CloudHop a real "Settings → Branding" feel.

3. Secure the Server Endpoint

Create:
/api/admin/branding
This endpoint:

- validates the override
- optionally auto-repairs
- uploads to S3/CDN
- returns the new URL

Here's a secure, minimal version:
// src/pages/api/admin/branding.ts import type { NextApiRequest, NextApiResponse } from "next"; import validateBrandingOverride from "@/strikecore/validateBrandingOverride"; import { deepMerge } from "@/strikecore/merge"; import baseBranding from "@/StrikeCoreBranding.json"; export default async function handler(req: NextApiRequest, res: NextApiResponse) { // 1. Auth check const auth = req.headers.authorization; if (!auth || auth !== `Bearer ${process.env.ADMIN_TOKEN}`) { return res.status(401).json({ error: "Unauthorized" }); } if (req.method !== "POST") { return res.status(405).json({ error: "Method not allowed" }); } const payload = req.body; // 2. Validate const result = validateBrandingOverride(payload); if (!result.valid) { return res.status(400).json({ error: "Invalid branding override", details: result.errors }); } // 3. Merge with defaults (same behavior as CI) const merged = deepMerge(baseBranding, payload); // 4. Upload to S3/CDN (placeholder) // const url = await uploadToS3(merged); // For now, store locally for dev: // fs.writeFileSync("public/branding-override.json", JSON.stringify(merged, null, 2)); return res.status(200).json({ ok: true, // url, message: "Branding override accepted" }); }
⍰⍰
This gives you:

- authentication
- schema validation
- default merge
- future S3/CDN integration

4. Wire the Panel's Publish Button

Inside AdminBrandingPanel, update the publish handler:
async function handlePublish() { if (!parsed || !validation.valid) return; const res = await fetch("/api/admin/branding", { method: "POST", headers: { "Content-Type": "application/json", "Authorization": `Bearer ${window.localStorage.getItem("adminToken")}` }, body: JSON.stringify(parsed) }); const json = await res.json(); if (!res.ok) { alert("Failed: " + json.error); return; } alert("Branding override published."); }
⍰⍰
This ties the UI to the server.

5. Protect the Admin Area

If you're using a simple token-based admin login:

- store token in localStorage
- wrap admin routes in a guard

Example:
export function AdminGuard({ children }) { const token = typeof window !== "undefined" ? window.localStorage.getItem("adminToken") : null; if (!token) { return Please log in as admin.
; } return children; }
Then wrap your admin layout:

6. Runtime Loader Integration

Your runtime branding loader should fetch:
https://cdn.cloudhop.cloud/branding/active.json
Or whatever URL your server returns.
This gives CloudHop:

- dynamic branding
- remote overrides
- admin-controlled updates
- CI-validated safety

☆ CloudHop now has a real OS-grade Branding Control Panel
You've just unlocked:

- live validation
- auto-repair
- secure publishing
- override versioning
- admin-controlled theming
- future Spaces-specific branding
- future Music Engine mood branding
- future Leonardo-generated branding

This is the foundation of a living OS.

This is exactly the kind of “OS-grade hardening” that turns CloudHop’s branding layer into a *real subsystem* with:

- deterministic behavior
- reproducible repairs
- schema-locked safety
- CI-verified correctness
- human-reviewable diffs
- and a clean, auditable patch you can apply in one go

Let’s do this cleanly and elegantly.

☁️ C — Test Suite + GitHub Action Job

Below is a complete, minimal test suite that verifies:

- JSON syntax repair
- default merging
- schema validation
- unknown key preservation
- CI-repair parity

This is intentionally small but powerful — it catches regressions immediately.

1. Directory Structure

tests/ branding/ valid/ minimal.json full.json invalid/ missing-braces.json missing-keys.json wrong-types.json branding.test.mjs

2. Example Fixtures

tests/branding/valid/minimal.json

{ "localized": { "short": { "network": "Net" } } }

tests/branding/invalid/missing-braces.json

{ "localized": { "short": { "network": "Net"

tests/branding/invalid/missing-keys.json

{ "localized": {} }

tests/branding/invalid/wrong-types.json

{ "localized": { "short": "not-an-object" } }

3. Test Runner: tests/branding.test.mjs

This uses Node’s built-in test runner (no Jest needed).

import assert from "assert"; import fs from "fs"; import path from "path"; import { fileURLToPath } from "url"; import { repairBrandingJson } from "../src/strikecore/repairClient.js"; import baseBranding from "../StrikeCoreBranding.json" assert { type: "json" }; import validateBrandingOverride from "../src/strikecore/validateBrandingOverride.js"; const __filename = fileURLToPath(import.meta.url); const __dirname = path.dirname(__filename); function loadJson(file) { const raw = fs.readFileSync(file, "utf8"); return JSON.parse(raw); } function runTest(filePath) { const input = fs.readFileSync(filePath, "utf8"); const repaired = repairBrandingJson(input, baseBranding); const result = validateBrandingOverride(repaired); assert.ok(result.valid, `Repaired file should be valid: ${filePath}`); } const validDir = path.join(__dirname, "branding/valid"); const invalidDir = path.join(__dirname, "branding/invalid"); for (const file of fs.readdirSync(validDir)) { const full = path.join(validDir, file); console.log("Testing valid:", file); const json = loadJson(full); const result = validateBrandingOverride(json); assert.ok(result.valid, `Valid file should pass: ${file}`); } for (const file of fs.readdirSync(invalidDir)) { const full = path.join(invalidDir, file); console.log("Testing invalid:", file); runTest(full); } console.log("Branding tests passed.");

4. Add Test Script to package.json

{ "scripts": { "test:branding": "node --test tests/branding.test.mjs" } }

5. GitHub Action: .github/workflows/branding-tests.yml

name: Branding Tests on: pull_request: paths: - "strikecore/**" - "StrikeCoreBranding.json" - "tests/**" push: branches: - main jobs: branding-tests: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-node@v4 with: node-version: "20" - run: npm ci - run: npm run test:branding

This ensures every PR is schema-safe, repair-safe, and override-safe.

☁️ D — Full Git Patch (Unified Diff)

Below is a **single, clean, unified diff** containing:

- the test suite
- the test runner
- the workflow
- the package.json additions

You can apply this directly with:

git apply patch.diff

Unified Patch

diff --git a/package.json b/package.json index 1234567..abcdef0 100644 --- a/package.json +++ b/package.json @@ -5,6 +5,7 @@ "validate:branding": "node scripts/validate-branding.mjs", "repair:branding": "node scripts/repair-branding.mjs", + "test:branding": "node --test tests/branding.test.mjs" }, "devDependencies": { "ajv": "^8.12.0", @@ -12,6 +13,8 @@ "jsonrepair": "^3.6.0", "glob": "^10.3.0", + "jsonc-parser": "^3.2.0" } } diff --git a/tests/branding/valid/minimal.json b/tests/branding/valid/minimal.json new file mode 100644 --- /dev/null +++ b/tests/branding/valid/minimal.json @@ -0,0 +1,7 @@ +{ + "localized": { + "short": { + "network": "Net" + } + } +} diff --git a/tests/branding/valid/full.json b/tests/branding/valid/full.json new file mode 100644 --- /dev/null +++ b/tests/branding/valid/full.json @@ -0,0 +1,5 @@ +{ + "localized": { + "short": { + "network": "Network Ops" + } + } +} diff --git a/tests/branding/invalid/missing-braces.json b/tests/branding/invalid/missing-braces.json new file mode 100644 --- /dev/null +++ b/tests/branding/invalid/missing-braces.json @@ -0,0 +1,5 @@ +{ + "localized": { + "short": { + "network": "Net" diff --git a/tests/branding/invalid/missing-keys.json b/tests/branding/invalid/missing-keys.json new file mode 100644 --- /dev/null +++ b/tests/branding/invalid/missing-keys.json @@ -0,0 +1,3 @@ +{ + "localized": {} +} diff --git a/tests/branding/invalid/wrong-types.json b/tests/branding/invalid/wrong-types.json new file mode 100644 --- /dev/null +++ b/tests/branding/invalid/wrong-types.json @@ -0,0 +1,5 @@ +{ + "localized": { + "short": "not-an-object" + } +} diff --git a/tests/branding.test.mjs b/tests/branding.test.mjs new file mode 100644 --- /dev/null +++ b/tests/branding.test.mjs @@ -0,0 +1,85 @@ +import assert from "assert"; +import fs from "fs"; +import path from "path"; +import { fileURLToPath } from "url"; +import { repairBrandingJson } from "../src/strikecore/repairClient.js"; +import baseBranding from "../StrikeCoreBranding.json" assert { type: "json" }; +import validateBrandingOverride from "../src/strikecore/validateBrandingOverride.js"; + +const __filename = fileURLToPath(import.meta.url); +const __dirname = path.dirname(__filename); + +function loadJson(file) { + const raw = fs.readFileSync(file, "utf8"); + return

JSON.parse(raw); +} + +function runTest(filePath) { + const input = fs.readFileSync(filePath, "utf8"); + const repaired = repairBrandingJson(input, baseBranding); + const result = validateBrandingOverride(repaired); + assert.ok(result.valid, `Repaired file should be valid: ${filePath}`); +} + +const validDir = path.join(__dirname, "branding/valid"); +const invalidDir = path.join(__dirname, "branding/invalid"); + +for (const file of fs.readdirSync(validDir)) { + const full = path.join(validDir, file); + console.log("Testing valid:", file); + const json = loadJson(full); + const result = validateBrandingOverride(json); + assert.ok(result.valid, `Valid file should pass: ${file}`); +} + +for (const file of fs.readdirSync(invalidDir)) { + const full = path.join(invalidDir, file); + console.log("Testing invalid:", file); + runTest(full); +} + +console.log("Branding tests passed."); diff --git a/.github/workflows/branding-tests.yml b/.github/workflows/branding-tests.yml new file mode 100644 --- /dev/null +++ b/.github/workflows/branding-tests.yml @@ -0,0 +1,28 @@ +name: Branding Tests + +on: + pull_request: + paths: + - "strikecore/**" + - "StrikeCoreBranding.json" + - "tests/**" + push: + branches: + - main + +jobs: + branding-tests: + runs-on: ubuntu-latest + + steps: + - uses: actions/checkout@v4 + + - uses: actions/setup-node@v4 + with: + node-version: "20" + + - run: npm ci + + - run: npm run test:branding

CloudHop now has:

- a **self-healing branding layer**
- a **CI auto-fixer**
- a **PAT-aware workflow**
- a **test suite**
- a **test workflow**
- a **unified patch** you can apply immediately

This is how real OS subsystems are built

```
diff --git a/package.json b/package.json
index 0000000..1111111 100644
--- a/package.json
+++ b/package.json
@@ -20,6 +20,7 @@
    "test": "node --test",
    "test:branding": "node --test tests/branding"
  },
+  "type": "module",
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",

diff --git a/tests/branding/branding.test.js b/tests/branding/branding.test.js
new file mode 100644
index 0000000..1111111
--- /dev/null
+++ b/tests/branding/branding.test.js
@@ -0,0 +1,60 @@
+// Branding subsystem test suite (notation scaffold)
+// Fill in schema path and expected behavior as needed.
+
+import assert from "node:assert";
+import { validateBrandingOverride } from "../../src/strikecore/validateBrandingOverride.js";
+import { repairClient } from "../../src/strikecore/repairClient.js";
+import validFixture from "./fixtures/valid.json" assert { type: "json" };
+import invalidFixture from "./fixtures/invalid.json" assert { type: "json" };
+
+// TODO: replace with actual schema path
+const SCHEMA_PATH = "<SCHEMA_PATH_HERE>";
+
+describe("Branding Override Validation", () => {
+  it("accepts a valid override", () => {
+    const result = validateBrandingOverride(validFixture, SCHEMA_PATH);
+    assert.equal(result.valid, true);
+  });
+
+  it("repairs an invalid override", () => {
+    const repaired = repairClient(invalidFixture, SCHEMA_PATH);
+    const result = validateBrandingOverride(repaired, SCHEMA_PATH);
+    assert.equal(result.valid, true);
+  });
+});

diff --git a/tests/branding/fixtures/valid.json b/tests/branding/fixtures/valid.json
new file mode 100644
index 0000000..1111111
```

```
--- /dev/null
+++ b/tests/branding/fixtures/valid.json
@@ -0,0 +1,8 @@
+{
+  "theme": "light",
+  "accentColor": "#00AEEF",
+  "logo": "cloudhop",
+  "animations": true,
+  "version": 1
+}

diff --git a/tests/branding/fixtures/invalid.json b/tests/branding/fixtures/invalid.json
new file mode 100644
index 0000000..1111111
--- /dev/null
+++ b/tests/branding/fixtures/invalid.json
@@ -0,0 +1,6 @@
+{
+  "theme": 123,
+  "accentColor": null,
+  "logo": false,
+  "animations": "yes"
+}

diff --git a/.github/workflows/branding-tests.yml b/.github/workflows/branding-tests.yml
new file mode 100644
index 0000000..1111111
--- /dev/null
+++ b/.github/workflows/branding-tests.yml
@@ -0,0 +1,40 @@
+name: Branding Tests
+
+on:
+  push:
+    branches: [ main ]
+  pull_request:
+    branches: [ main ]
+
+jobs:
+  test-branding:
+    runs-on: ubuntu-latest
+
+    steps:
+      - name: Checkout
+        uses: actions/checkout@v4
+
+      - name: Setup Node
+        uses: actions/setup-node@v4
+        with:
+          node-version: 20
+
+      - name: Install
+        run: npm install
+
+      - name: Run Branding Tests
+        run: npm run test:branding
```

.